

1)Design a "My Hobbies" webpage with: A well-structured layout using semantic tags.

Lists of hobbies and favorite books/movies. A form to collect user feedback. Multimedia elements,

like an image or video showcasing a hobby.(WITH VIDEOS AND IMAGES)

```
<!DOCTYPE html>
<html>
<head>
  <title>My Hobbies</title>
</head>
<body style="font-family: Arial; background-color: #f0f0f0; margin: 0; padding: 0;">

  <header style="background-color: #4CAF50; color: white; padding: 20px; text-align: center;">
    <h1>My Hobbies</h1>
    <p>All the things I love to do!</p>
  </header>

  <nav style="background-color: #ddd; padding: 10px; text-align: center;">
    <a href="#hobbies" style="margin: 10px;">Hobbies</a>
    <a href="#favorites" style="margin: 10px;">Favorites</a>
    <a href="#media" style="margin: 10px;">Gallery</a>
    <a href="#feedback" style="margin: 10px;">Feedback</a>
  </nav>

  <main style="padding: 20px;">

    <section id="hobbies">
      <h2>My Hobbies</h2>
      <ul>
        <li>Drawing</li>
        <li>Listening to music</li>
        <li>Playing cricket</li>
        <li>Coding simple programs</li>
        <li>Watching science videos</li>
      </ul>
    </section>

    <section id="favorites">
      <h2>My Favorite Books and Movies</h2>
      <h3>Books:</h3>
      <ul>
        <li>Harry Potter</li>
        <li>Wings of Fire</li>
```

```
<li>The Secret</li>
</ul>
```

```
<h3>Movies:</h3>
<ul>
  <li>Interstellar</li>
  <li>3 Idiots</li>
  <li>Avengers: Endgame</li>
</ul>
</section>
```

```
<section id="media">
  <h2>Gallery</h2>
  <p>This is me playing guitar:</p>
  <video width="320" height="240" controls>
    <source src="guitar.mp4" type="video/mp4">
    Your browser does not support the video tag.
  </video>
```

```
<p>One of my drawings:</p>

</section>
```

```
<section id="feedback">
  <h2>Feedback Form</h2>
  <form>
    <label>Your Name:</label><br>
    <input type="text" name="name"><br><br>

    <label>Your Feedback:</label><br>
    <textarea rows="5" cols="40" name="message"></textarea><br><br>

    <input type="submit" value="Submit">
  </form>
</section>
```

```
</main>
```

```
<footer style="background-color: #4CAF50; color: white; text-align: center; padding: 10px;">
  <p>Made by [Your Name] - Student Project</p>
</footer>
```

```
</body>
```

</html>

Design a "My Hobbies" webpage with: A well-structured layout using semantic tags.

Lists of hobbies and favorite books/movies. A form to collect user feedback. Multimedia elements,

like an image or video showcasing a hobby.(WITHOUT VIDEOS AND IMAGES)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My Hobbies</title>
```

```
</head>
```

```
<body style="font-family: Arial; background-color: #f9f9f9; padding: 20px;">
```

```
  <header style="background-color: #2196F3; color: white; padding: 15px; text-align: center;">
```

```
    <h1>My Hobbies</h1>
```

```
    <p>Things I enjoy doing in my free time!</p>
```

```
  </header>
```

```
  <nav style="background-color: #eee; padding: 10px; text-align: center;">
```

```
    <a href="#hobbies" style="margin: 10px;">Hobbies</a>
```

```
    <a href="#favorites" style="margin: 10px;">Favorites</a>
```

```
    <a href="#feedback" style="margin: 10px;">Feedback</a>
```

```
  </nav>
```

```
  <main>
```

```
    <section id="hobbies" style="margin-top: 20px;">
```

```
      <h2>My Hobbies</h2>
```

```
      <ul>
```

```
        <li>Reading novels</li>
```

```
        <li>Writing short stories</li>
```

```
        <li>Playing badminton</li>
```

```
        <li>Learning new coding languages</li>
```

```
        <li>Spending time with friends</li>
```

```
      </ul>
```

```
    </section>
```

```
    <section id="favorites" style="margin-top: 20px;">
```

```
      <h2>Favorite Books and Movies</h2>
```

```
      <h3>Books:</h3>
```

```
      <ul>
```

```
        <li>The Alchemist</li>
```

```
<li>Harry Potter Series</li>
<li>Atomic Habits</li>
</ul>
```

```
<h3>Movies:</h3>
<ul>
  <li>3 Idiots</li>
  <li>Inception</li>
  <li>The Pursuit of Happyness</li>
</ul>
</section>
```

```
<section id="feedback" style="margin-top: 20px;">
  <h2>Feedback Form</h2>
  <form>
    <label>Your Name:</label><br>
    <input type="text" name="name"><br><br>

    <label>Your Feedback:</label><br>
    <textarea name="feedback" rows="4" cols="40"></textarea><br><br>

    <input type="submit" value="Submit">
  </form>
</section>
```

```
</main>
```

```
<footer style="background-color: #2196F3; color: white; text-align: center; padding: 10px;
margin-top: 20px;">
  <p>Made by [Your Name] | Student Project</p>
</footer>
```

```
</body>
</html>
```

2. Design a structured HTML5 page using semantic tags (<header>, <main>, <footer>).

A navigation bar with links to sections: About Me, Projects, and Contact.

A list of skills using an unordered list.

A contact form with fields for name, email, and message.

Use an image as a profile picture.

Add a table to display project details (e.g., project name, description, and status).

```
<!DOCTYPE html>
```

```

<html>
<head>
  <title>My Portfolio</title>
</head>
<body style="font-family: Arial; background-color: #f0f0f0; padding: 0; margin: 0;">

  <header style="background-color: #333; color: white; padding: 20px; text-align: center;">
    <h1>Welcome to My Portfolio</h1>
    <p>Student | Learner | Coder</p>
  </header>

  <nav style="background-color: #ddd; padding: 10px; text-align: center;">
    <a href="#about" style="margin: 10px;">About Me</a>
    <a href="#projects" style="margin: 10px;">Projects</a>
    <a href="#contact" style="margin: 10px;">Contact</a>
  </nav>

  <main style="padding: 20px;">

    <section id="about">
      <h2>About Me</h2>
      
      <p>Hello! I'm a student passionate about technology, coding, and building fun projects.</p>

      <h3>My Skills</h3>
      <ul>
        <li>HTML & CSS</li>
        <li>C Programming</li>
        <li>C++</li>
        <li>Python (Basics)</li>
        <li>Problem Solving</li>
      </ul>
    </section>

    <section id="projects" style="margin-top: 30px;">
      <h2>My Projects</h2>
      <table border="1" cellpadding="10" cellspacing="0" style="background-color: white;">
        <tr>
          <th>Project Name</th>
          <th>Description</th>
          <th>Status</th>
        </tr>

```


</html>

3. Design an 8x8 checkerboard pattern where alternating cells are black and white. Add a hover effect that

changes the background color of any square when the mouse is over it.

<!DOCTYPE html>

<html>

<head>

<title>Checkerboard</title>

<style>

```
.board {  
  display: grid;  
  grid-template-columns: repeat(8, 60px);  
  grid-template-rows: repeat(8, 60px);  
  gap: 0;  
  width: 480px;  
  margin: 50px auto;  
  border: 2px solid black;  
}
```

```
.square {  
  width: 60px;  
  height: 60px;  
}
```

```
.black {  
  background-color: black;  
}
```

```
.white {  
  background-color: white;  
}
```

```
.square:hover {  
  background-color: yellow;  
}
```

</style>

</head>

<body>

<h2 style="text-align:center;">8x8 Checkerboard</h2>

```

<div class="board">
  <!-- JavaScript will fill this -->
</div>

<script>
  const board = document.querySelector('.board');
  for (let row = 0; row < 8; row++) {
    for (let col = 0; col < 8; col++) {
      const square = document.createElement('div');
      square.classList.add('square');

      if ((row + col) % 2 === 0) {
        square.classList.add('white');
      } else {
        square.classList.add('black');
      }

      board.appendChild(square);
    }
  }
</script>

</body>
</html>

```

4. Create a page with:

- A heading that reads "VIBGYOR IMPLEMENTATION" styled appropriately.
- A set of vertically aligned, evenly spaced colored columns (matching the VIBGYOR sequence).
- Add hover effects to change the brightness of each column when hovered over.

```

<!DOCTYPE html>
<html>
<head>
  <title>VIBGYOR Implementation</title>
  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
    }

    h1 {

```



```
text-align: center;
padding: 20px;
background-color: #333;
color: white;
font-size: 32px;
}
```

```
.vibgyor-container {
display: flex;
flex-direction: column;
align-items: center;
gap: 10px;
padding: 30px;
}
```

```
.color-box {
width: 300px;
height: 50px;
transition: filter 0.3s;
}
```

```
.color-box:hover {
filter: brightness(120%);
}
```

```
.violet { background-color: violet; }
.indigo { background-color: indigo; }
} .blue { background-color: blue; }
.green { background-color: green; }
.yellow { background-color: yellow; }
.orange { background-color: orange; }
} .red { background-color: red; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>VIBGYOR IMPLEMENTATION</h1>
```

```
<div class="vibgyor-container">
  <div class="color-box violet"></div>
<div class="color-box indigo"></div>
<div class="color-box blue"></div>
<div class="color-box green"></div>
```

```
<div class="color-box yellow"></div>
<div class="color-box orange"></div>
<div class="color-box red"></div>
</div>
```

```
</body>
</html>
```

5. Admission Enquiry Form

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Admission Enquiry Form</title>
```

```
</head>
```

```
<body style="font-family: Arial; background-color: #f2f2f2; padding: 20px;">
```

```
  <h2 style="text-align: center; color: #333;">Admission Enquiry Form</h2>
```

```
  <form style="max-width: 500px; margin: auto; background: white; padding: 20px;
border-radius: 8px;">
```

```
    <label for="name">Full Name:</label><br>
```

```
    <input type="text" id="name" name="name" style="width: 100%; padding: 8px;"
required><br><br>
```

```
    <label for="email">Email ID:</label><br>
```

```
    <input type="email" id="email" name="email" style="width: 100%; padding: 8px;"
required><br><br>
```

```
    <label for="phone">Phone Number:</label><br>
```

```
    <input type="tel" id="phone" name="phone" style="width: 100%; padding: 8px;"
required><br><br>
```

```
    <label for="course">Course Interested In:</label><br>
```

```
    <input type="text" id="course" name="course" style="width: 100%; padding: 8px;"
required><br><br>
```

```
    <label for="message">Additional Message:</label><br>
```

```
    <textarea id="message" name="message" rows="4" style="width: 100%; padding:
8px;"></textarea><br><br>
```

```
    <input type="submit" value="Submit Enquiry" style="padding: 10px 20px; background-color:
#4CAF50; color: white; border: none; border-radius: 4px; cursor: pointer;">
```

</form>

</body>

</html>

6. Design Student Course Registration Form:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8" />
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
```

```
  <title>CBIT Course Registration Form</title>
```

```
  <style>
```

```
    body {
```

```
      font-family: Arial, sans-serif;
```

```
      margin: 40px;
```

```
      background: #fff;
```

```
      color: #000;
```

```
    }
```

```
    h1, h2 {
```

```
      text-align: center;
```

```
    }
```

```
    .header {
```

```
      text-align: center;
```

```
      font-weight: bold;
```

```
    }
```

```
    .section {
```

```
      margin-top: 20px;
```

```
    }
```

```
    label {
```

```
      display: inline-block;
```

```
      width: 250px;
```

```
      margin-bottom: 10px;
```

```
    }
```

```
    input[type="text"], input[type="email"], input[type="tel"], input[type="date"] {
```

```
      width: 300px;
```

```
      padding: 5px;
```

```

    margin-bottom: 10px;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
}

th, td {
    border: 1px solid #000;
    padding: 8px;
    text-align: center;
}

.footer {
    margin-top: 30px;
    font-size: 14px;
}

.signatures {
    display: flex;
    justify-content: space-between;
    margin-top: 40px;
}
</style>
</head>
<body>

<div class="header">
    <h2>CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)</h2> <p>Chaitanya
    Bharathi P.O., Kokapet (V), Gandipet (M), Hyderabad – 500 075, Telangana State</p>
    <p>Phone No: 040-24193280; Fax No: 040 – 24193278; Website: www.cbit.ac.in</p>
    <h3>ACADEMIC & EXAMINATION CELL</h3>
    <h2>REGISTRATION FORM</h2>
</div>

<div class="section">
    <label for="date">Date:</label>
    <input type="date" id="date" name="date"><br>

    <label for="studentName">1. Name of the Student:</label>

```

<input type="text" id="studentName" name="studentName">

<label for="rollNo">2. Roll No.:</label>

<input type="text" id="rollNo" name="rollNo">

<label for="programme">3. Name of the Programme:</label>

<input type="text" id="programme" name="programme" placeholder="B.E. / B.Tech. (CSE, ECE...)">

<label for="semester">4. Semester:</label>

<input type="text" id="semester" name="semester" placeholder="e.g., III">

<label for="mobile">5. Mobile No.:</label>

<input type="tel" id="mobile" name="mobile">

<label for="email">6. Email ID:</label>

<input type="email" id="email" name="email">

</div>

<div class="section">

<h4>7. Details of Annual Tuition Fee: (Attach Proofs for Payment)</h4>

<label>a) Status of Tuition Fee Payment:</label>

<input type="text">

<label>b) Date of Fee Payment for the A.Y. 2024-25:</label>

<input type="text">

<label>c) Mode of Payment:</label>

<input type="text">

<label>d) Receipt No. with date:</label>

<input type="text">

</div>

<div class="section">

<h4>8. Details of Subjects Registering for III Semester:</h4>

<table>

<thead>

<tr>

<th>No.</th>

<th>Course Code</th>

<th>Name of the Course</th>

<th>Core / Programme Elective / Internship</th>

<th>Pursuing through Institute / NPTEL / MOOCs</th>

<th>Remarks</th>

</tr>

</thead>

```

<tbody>
  <!-- Repeat rows as needed -->
  <tr><td>1</td><td></td><td></td><td></td><td></td><td></td></tr>
  <tr><td>2</td><td></td><td></td><td></td><td></td><td></td></tr>
  <tr><td>3</td><td></td><td></td><td></td><td></td><td></td></tr>
  <tr><td>4</td><td></td><td></td><td></td><td></td><td></td></tr>
  <tr><td>5</td><td></td><td></td><td></td><td></td><td></td></tr>
</tbody>
</table>
</div>
<div class="footer">
  <p><strong>Note:</strong> Registration is not permitted with any fee due.</p>

  <div class="signatures">
    <p>Signature of the Student</p>
    <p>Signature of the HOD</p>
  </div>

  <div style="margin-top: 20px;">
    <p><strong>Remarks of Head of the Department:</strong></p>
    <p>Fee Due, if any: _____</p>
  </div>

  <div class="signatures" style="margin-top: 40px;">
    <p><strong>DIRECTOR – AEC & COE</strong></p>
    <p><strong>PRINCIPAL</strong></p>
  </div>
</div>

</body>
</html>

```

7. Design CBIT Student Admission Form

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CBIT Student Admission Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 40px;
    }
  </style>

```

```

    background-color: #fff;
}
h1, h2, h3 {
    text-align: center;
}
.section {
    margin-top: 20px;
}
label {
    display: inline-block;
    width: 250px;
    margin: 8px 0;
}
input[type="text"], input[type="email"], input[type="date"], input[type="tel"] {
    width: 300px;
    padding: 5px;
}
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
}
th, td {
    border: 1px solid #000;
    padding: 8px;
    text-align: left;
}
.checkbox-col {
    text-align: center;
}
.footer {
    margin-top: 30px;
}
</style>
</head>
<body>

```

<h1>CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)</h1>

<h3>ACADEMIC AND EXAMINATION CELL</h3>

<h2>STUDENT ADMISSION FORM</h2>

<div class="section">

<label for="name">Name of Student:</label>
<input type="text" id="name" name="name">

<label for="roll">TSEAMCET Hall Ticket No.:</label>
<input type="text" id="roll" name="roll">

<label for="email">Email ID:</label>
<input type="email" id="email" name="email">

<label for="mobile">Mobile No.:</label>
<input type="tel" id="mobile" name="mobile">

<label for="dob">Date of Birth:</label>
<input type="date" id="dob" name="dob">

<label for="category">Category (OC/BC/SC/ST):</label>
<input type="text" id="category" name="category">

<label for="branch">Branch Allotted:</label>
<input type="text" id="branch" name="branch">

</div>

<div class="section">
<h3>Documents Verification Checklist</h3>
<table>
<thead>
<tr>
<th>Sl. No.</th>
<th>Certificate Name</th>
<th class="checkbox-col">Submitted</th>
</tr>
</thead>
<tbody>
<tr><td>1</td><td>Allotment Order (Final Phase)</td><td class="checkbox-col"><input type="checkbox"></td></tr>
<tr><td>2</td><td>Joining Report (Final Phase)</td><td class="checkbox-col"><input type="checkbox"></td></tr>
<tr><td>3</td><td>Convenor Fee Paid Challan (if applicable)</td><td class="checkbox-col"><input type="checkbox"></td></tr>
<tr><td>4</td><td>TSEAMCET Hall Ticket</td><td class="checkbox-col"><input type="checkbox"></td></tr>
<tr><td>5</td><td>TSEAMCET Rank Card</td><td class="checkbox-col"><input


```

type="checkbox"></td></tr>
    <tr><td>6</td><td>Receipt of Certificates from TSEAMCET</td><td
class="checkbox-col"><input type="checkbox"></td></tr>
    <tr><td>7</td><td>S.S.C or equivalent Marks Memo</td><td class="checkbox-col"><input
type="checkbox"></td></tr>
    <tr><td>8</td><td>Inter or equivalent Memo</td><td class="checkbox-col"><input
type="checkbox"></td></tr>
    <tr><td>9</td><td>Study Certificates (VI to XII)</td><td class="checkbox-col"><input
type="checkbox"></td></tr>
    <tr><td>10</td><td>Transfer Certificate (TC)</td><td class="checkbox-col"><input
type="checkbox"></td></tr>
    <tr><td>11</td><td>EWS Income Certificate (if applicable)</td><td
class="checkbox-col"><input type="checkbox"></td></tr>
    <tr><td>12</td><td>Caste Certificate (if applicable)</td><td class="checkbox-col"><input
type="checkbox"></td></tr>
    <tr><td>13</td><td>Other Certificates (PHC, NCC, CAP, Sports)</td><td
class="checkbox-col"><input type="checkbox"></td></tr>
    <tr><td>14</td><td>Aadhaar Card</td><td class="checkbox-col"><input
type="checkbox"></td></tr>
    <tr><td>15</td><td>5 Passport Size Photos</td><td class="checkbox-col"><input
type="checkbox"></td></tr>
</tbody>
</table>
</div>

<div class="footer">
    <p><strong>Note:</strong> Candidates must bring original certificates and two Xerox
sets.</p>
    <p><strong>Signature of Student:</strong> _____</p>
    <p><strong>Signature of Verifying Officer:</strong> _____</p>
</div>

</body>
</html>

```

8. Design static website using HTML, CSS and Bootstrap.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>My Static Website</title>

```

```

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
<link rel="stylesheet" href="style.css" />
<style>
  .hero {
    background: linear-gradient(to right, #007bff, #6610f2);
    height: 100vh;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-direction: column;
    text-align: center;
  }
  .card img {
    height: 200px;
    object-fit: cover;
  }
</style>
</head>
<body>

<!-- Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">MySite</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
      data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false"
      aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link active" href="#">Home</a></li> <li
        class="nav-item"><a class="nav-link" href="#about">About</a></li> <li
        class="nav-item"><a class="nav-link" href="#services">Services</a></li> <li
        class="nav-item"><a class="nav-link" href="#contact">Contact</a></li> </ul>
      </div>
    </div>
  </nav>

<!-- Hero Section -->
<header class="hero">

```

```

<div class="container">
  <h1>Welcome to My Static Website</h1>
  <p class="lead">Simple. Elegant. Responsive.</p>
  <a href="#about" class="btn btn-primary">Learn More</a>
</div>
</header>

<!-- About Section -->
<section id="about" class="py-5">
  <div class="container">
    <h2>About Me</h2>
    <p>This is a static website built using HTML, CSS, and Bootstrap. It's fully responsive and
easy to maintain.</p>
  </div>
</section>
<!-- Services Section -->
<section id="services" class="bg-light py-5">
  <div class="container">
    <h2 class="mb-4">Services</h2>
    <div class="row">
      <div class="col-md-4">
        <div class="card">
          
          <div class="card-body">
            <h5 class="card-title">Web Design</h5>
            <p class="card-text">Creative and responsive web design solutions tailored to your
needs.</p>
          </div>
        </div>
      </div>
      <div class="col-md-4">
        <div class="card">
          
          <div class="card-body">
            <h5 class="card-title">Development</h5>
            <p class="card-text">Custom web development using latest technologies and best
practices.</p>
          </div>
        </div>
      </div>
      <div class="col-md-4">
        <div class="card">

```

```
        
        <div class="card-body">
            <h5 class="card-title">SEO</h5>
            <p class="card-text">Improve your website's visibility and ranking with our SEO
strategies.</p>
        </div>
    </div>
</div>
</div>
</div>
</div>
</div>
</section>
```

<!-- Contact Section -->

```
<section id="contact" class="py-5">
    <div class="container">
        <h2>Contact</h2>
        <form>
            <div class="mb-3">
                <label for="name" class="form-label">Name</label>
                <input type="text" class="form-control" id="name" required>
            </div>
            <div class="mb-3">
                <label for="email" class="form-label">Email</label>
                <input type="email" class="form-control" id="email" required>
            </div>
            <div class="mb-3">
                <label for="message" class="form-label">Message</label>
                <textarea class="form-control" id="message" rows="4" required></textarea>
            </div>
            <button type="submit" class="btn btn-primary">Send</button>
        </form>
    </div>
</section>
```

<!-- Footer -->

```
<footer class="bg-dark text-white text-center py-3">
    <p>&copy; 2025 My Static Website. All rights reserved.</p>
</footer>
```

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
```

</html>

9. Updating a Product Price on an E-Commerce Website by navigating to a particular element using DOM .

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>E-Commerce Product Page</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f8f9fa;
    }
    .product-card {
      max-width: 500px;
      margin: 50px auto;
      padding: 30px;
      border: 1px solid #ddd;
      border-radius: 10px;
      background-color: #fff;
      box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    }
    .product-image {
      width: 100%;
      height: auto;
      border-radius: 8px;
    }
    .price {
      color: green;
      font-size: 24px;
      font-weight: bold;
    }
    footer {
      margin-top: 50px;
      padding: 20px;
      text-align: center;
```

```

        background-color: #343a40;
        color: white;
    }
</style>
</head>
<body>

<!-- Product Card -->
<div class="product-card text-center">
    <h2>Wireless Headphones</h2>
    
    <p>High-quality wireless headphones with noise cancellation and long battery life.</p> <p
    class="price" id="product-price">$99.99</p>
    <button class="btn btn-primary" onclick="updatePrice()">Update Price</button>
</div>

<!-- Footer -->
<footer>
    &copy; 2025 My E-Commerce Store. All rights reserved.
</footer>
<!-- JavaScript -->
<script>
    function updatePrice() {
        const priceElement = document.getElementById("product-price");
        priceElement.textContent = "$79.99";
    }
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

10. Update Real-Time Stock Price in a Finance Website using DOM.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Real-Time Stock Price</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"

```

```
rel="stylesheet">
<style>
  body {
    font-family: Arial, sans-serif;
    background-color: #f1f1f1;
  }
  .stock-container {
    max-width: 600px;
    margin: 50px auto;
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 4px 10px rgba(0,0,0,0.1);
  }
  .stock-price {
    font-size: 32px;
    color: green;
    font-weight: bold;
  }
  footer {
    margin-top: 50px;
    padding: 20px;
    text-align: center;
    background-color: #343a40;
    color: white;
  }
</style>
</head>
<body>

<div class="stock-container text-center">
  <h2>Live Stock Price</h2>
  <p>Company: <strong>ABC Corp</strong></p>
  <p class="stock-price" id="stock-price">$120.00</p>
  <button class="btn btn-success" onclick="updateStockPrice()">Update Price</button>
</div>

<footer>
  &copy; 2025 My Finance Site. All rights reserved.
</footer>

<script>
```

```

function updateStockPrice() {
  const priceElement = document.getElementById("stock-price");
  const newPrice = (100 + Math.random() * 50).toFixed(2); // Simulate live price update
  priceElement.textContent = `$$${newPrice}`;
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

11.Modify Live Score Update in a Sports Website using DOM

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Live Sports Score</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      background-color: #e9ecef;
      font-family: 'Segoe UI', sans-serif;
    }
    .scoreboard {
      max-width: 600px;
      margin: 60px auto;
      padding: 40px;
      background: #fff;
      border-radius: 12px;
      box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
      text-align: center;
    }
    .team-name {
      font-size: 24px;
      font-weight: 600;
    }
    .score {
      font-size: 40px;

```



```

    font-weight: bold;
    color: #dc3545;
}
footer {
    margin-top: 60px;
    padding: 20px;
    background: #343a40;
    color: white;
    text-align: center;
}
</style>
</head>
<body>

<div class="scoreboard">
    <h2>Live Match Score</h2>
    <p class="team-name">India vs Australia</p>
    <p class="score" id="live-score">120/3 (15.2 overs)</p>
    <button class="btn btn-danger" onclick="updateScore()">Update Score</button>
</div>

<footer>
    &copy; 2025 Sports Live. All rights reserved.
</footer>

<script>
function updateScore() {
    const scoreElement = document.getElementById("live-score");
    const runs = Math.floor(100 + Math.random() * 100);
    const wickets = Math.floor(Math.random() * 10);
    const overs = (Math.floor(Math.random() * 20) + Math.random()).toFixed(1);
    scoreElement.textContent = `${runs}/${wickets} (${overs} overs)`;
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

12. Locating Mouse coordinates using JavaScript.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mouse Coordinates Tracker</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body, html {
      height: 100%;
      margin: 0;
      display: flex;
      justify-content: center;
      align-items: center;
      background-color: #f0f0f0;
      font-family: Arial, sans-serif;
    }
    .tracker {
      text-align: center;
      padding: 40px;
      background: white;
      border-radius: 12px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
    }
    .coords {
      font-size: 24px;
      margin-top: 20px;
      color: #007bff;
    }
  </style>
</head>
<body>

  <div class="tracker">
    <h2>Mouse Coordinates</h2>
    <p class="coords" id="coordinates">Move your mouse...</p>
  </div>

  <script>
    document.addEventListener("mousemove", function(event) {
```

```

    const x = event.clientX;
    const y = event.clientY;
    document.getElementById("coordinates").textContent = `X: ${x}, Y: ${y}`;
  });
</script>

</body>
</html>

```

13. Create Dynamic Table content using Dynamic Document in Javascript.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dynamic Table Website</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px;
      background-color: #f5f5f5;
    }
    header, footer {
      background-color: #004080;
      color: white;
      text-align: center;
      padding: 15px 0;
    }
    main {
      background: white;
      padding: 20px;
      margin-top: 10px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }
    input, button {
      padding: 8px;
      margin: 5px;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin-top: 20px;

```

```

    }
    th, td {
        border: 1px solid #999;
        padding: 10px;
        text-align: left;
    }
    th {
        background-color: #ddd;
    }
</style>
</head>
<body>

<header>
    <h1>Dynamic Table Generator</h1>
</header>

<main>
    <h2>Add Row to Table</h2>
    <form id="dataForm">
        <input type="text" id="name" placeholder="Enter Name" required>
        <input type="text" id="course" placeholder="Enter Course" required>
        <input type="text" id="status" placeholder="Enter Status" required>
        <button type="submit">Add Row</button>
    </form>

    <h2>Table Content</h2>
    <table id="dataTable">
        <thead>
            <tr>
                <th>Sl. No.</th>
                <th>Name</th>
                <th>Course</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody>
            <!-- Rows will appear here -->
        </tbody>
    </table>
</main>

```

```
<footer>
  <p>Made by Student | © 2025</p>
</footer>
```

```
<script>
  let rowCount = 1;
  const form = document.getElementById("dataForm");
  const table = document.getElementById("dataTable").getElementsByTagName('tbody')[0];

  form.addEventListener("submit", function(e) {
    e.preventDefault();

    // Get input values
    const name = document.getElementById("name").value;
    const course = document.getElementById("course").value;
    const status = document.getElementById("status").value;

    // Insert row into table
    const newRow = table.insertRow();
    newRow.innerHTML = `
      <td>${rowCount++}</td>
      <td>${name}</td>
      <td>${course}</td>
      <td>${status}</td>
    `;

    // Clear form
    form.reset();
  });
</script>
```

```
</body>
</html>
```

14. Stacking Elements using Javascript.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Stacking Elements Using JavaScript</title>
  <style>
    body {
      font-family: Arial, sans-serif;
```

```
margin: 30px;
background-color: #f0f0f0;
}
header, footer {
background-color: #4CAF50;
color: white;
padding: 15px;
text-align: center;
}
main {
padding: 20px;
background: white;
box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
#stackArea {
margin-top: 20px;
min-height: 300px;
padding: 10px;
border: 2px dashed #ccc;
display: flex;
flex-direction: column;
gap: 10px;
background-color: #fafafa;
}
.stack-box {
width: 100%;
height: 50px;
background-color: #007BFF;
color: white;
text-align: center;
line-height: 50px;
font-weight: bold;
border-radius: 5px;
transition: transform 0.2s;
}
.stack-box:hover {
transform: scale(1.05);
background-color: #0056b3;
}
button {
padding: 10px 20px;
background-color: #4CAF50;
```

```

    color: white;
    border: none;
    cursor: pointer;
    border-radius: 5px;
  }
  button:hover {
    background-color: #45a049;
  }
</style>
</head>
<body>

<header>
  <h1>Stacking Elements with JavaScript</h1>
</header>

<main>
  <button onclick="addBox()">Add Stack Box</button>

  <div id="stackArea"></div>
</main>

<footer>
  <p>Made by Student | © 2025</p>
</footer>

<script>
  let count = 1;
  function addBox() {
    const stackArea = document.getElementById("stackArea");
    const box = document.createElement("div");
    box.className = "stack-box";
    box.innerText = "Box " + count++;
    stackArea.appendChild(box);
  }
</script>
</body>
</html>

```

15. Moving the Elements using Javascript

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Move the Box using JavaScript</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin: 30px;
      background-color: #f0f0f0;
    }

    header, footer {
      background-color: #6200EA;
      color: white;
      padding: 15px;
    }

    main {
      background: white;
      padding: 20px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
      display: inline-block;
    }

    #box {
      width: 50px;
      height: 50px;
      background-color: red;
      position: relative;
      margin: 20px auto;
      transition: all 0.3s ease;
    }

    button {
      padding: 10px;
      margin: 5px;
      background-color: #6200EA;
      color: white;
      border: none;
      cursor: pointer;
      border-radius: 4px;
```



```
}

button:hover {
  background-color: #3700B3;
}
</style>
</head>
<body>

<header>
  <h1>Move the Box</h1>
</header>

<main>
  <div id="box"></div>

  <div>
    <button onclick="move('up')">Up</button>
    <button onclick="move('left')">Left</button>
    <button onclick="move('right')">Right</button>
    <button onclick="move('down')">Down</button>
  </div>
</main>

<footer>
  <p>Made by Student | © 2025</p>
</footer>

<script>
  let x = 0, y = 0;

  function move(direction) {
    const box = document.getElementById("box");

    const step = 20; // how much to move per click

    switch (direction) {
      case 'up':
        y -= step;
        break;
      case 'down':
        y += step;
```

```

        break;
    case 'left':
        x -= step;
        break;
    case 'right':
        x += step;
        break;
    }

    box.style.transform = `translate(${x}px, ${y}px)`;
}
</script>

</body>
</html>

```

16. Explore the new features introduced in ES5 to recent.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Explore JavaScript Features (ES5 - ES14)</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      background-color: #f0f4f8;
    }
    header, footer {
      background-color: #2e3a59;
      color: white;
      text-align: center;
      padding: 20px;
    }
    main {
      padding: 30px;
    }
    section {
      background: white;
      margin-bottom: 20px;
      padding: 20px;
      border-radius: 10px;
    }
  </style>

```

```

    box-shadow: 0 0 10px rgba(0,0,0,0.1);
  }
  h2 {
    color: #2e3a59;
  }
  pre {
    background-color: #eee;
    padding: 10px;
    border-radius: 5px;
    overflow-x: auto;
  }
  button {
    margin-top: 10px;
    padding: 8px 12px;
    background-color: #2e3a59;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
  }
  button:hover {
    background-color: #1c253a;
  }
  .output {
    margin-top: 10px;
    color: green;
    font-weight: bold;
  }
</style>
</head>
<body>

<header>
  <h1>JavaScript Features (ES5 - ES14)</h1>
  <p>Click the buttons to see features in action!</p>
</header>

<main>

<section>
  <h2>ES6: Arrow Functions & Template Literals</h2>
  <pre>

```

```
const greet = (name) => `Hello, ${name}!`;
</pre>
<button onclick="runES6()">Run</button>
<div class="output" id="es6Output"></div>
</section>
```

```
<section>
  <h2>ES8: Async/Await</h2>
  <pre>
async function getData() {
  return await Promise.resolve("Data Loaded!");
}
  </pre>
  <button onclick="runES8()">Run</button>
  <div class="output" id="es8Output"></div>
</section>
```

```
<section>
  <h2>ES11: Optional Chaining & Nullish Coalescing</h2>
  <pre>
const user = {};
const name = user?.info?.name ?? "Guest";
  </pre>
  <button onclick="runES11()">Run</button>
  <div class="output" id="es11Output"></div>
</section>
```

```
<section>
  <h2>ES13: Array.at()</h2>
  <pre>
const items = [10, 20, 30];
const last = items.at(-1);
  </pre>
  <button onclick="runES13()">Run</button>
  <div class="output" id="es13Output"></div>
</section>
```

```
<section>
  <h2>ES14: findLast()</h2>
  <pre>
const nums = [1, 3, 5, 8, 10];
const lastEven = nums.findLast(n => n % 2 === 0);
  </pre>
  <button onclick="runES14()">Run</button>
  <div class="output" id="es14Output"></div>
</section>
```

```
</pre>
<button onclick="runES14()">Run</button>
<div class="output" id="es14Output"></div>
</section>
</main>

<footer>
  <p>Created by Student | © 2025</p>
</footer>

<script>
function runES6() {
  const greet = (name) => `Hello, ${name}!`;
  document.getElementById("es6Output").textContent = greet("Saneela");
}

async function runES8() {
  const getData = async () => await Promise.resolve("Data Loaded!");
  const result = await getData();
  document.getElementById("es8Output").textContent = result;
}

function runES11() {
  const user = {};
  const name = user?.info?.name ?? "Guest";
  document.getElementById("es11Output").textContent = name;
}

function runES13() {
  const items = [10, 20, 30];
  document.getElementById("es13Output").textContent = items.at(-1);
}

function runES14() {
  const nums = [1, 3, 5, 8, 10];
  const lastEven = nums.findLast(n => n % 2 === 0);
  document.getElementById("es14Output").textContent = lastEven;
}
</script>

</body>
</html>
```

17. Display multiplication table for the given number.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Multiplication Table Generator</title>
  <style>
    body {
      font-family: 'Segoe UI', sans-serif;
      background-color: #f4f4f4;
      margin: 0;
      padding: 0;
      text-align: center;
    }

    header, footer {
      background-color: #007BFF;
      color: white;
      padding: 15px;
    }

    main {
      padding: 30px;
    }

    input[type="number"] {
      padding: 10px;
      font-size: 16px;
      width: 200px;
    }

    button {
      padding: 10px 15px;
      font-size: 16px;
      background-color: #007BFF;
      color: white;
      border: none;
      margin-left: 10px;
      cursor: pointer;
      border-radius: 4px;
    }
```

```
button:hover {
  background-color: #0056b3;
}

.output {
  margin-top: 20px;
  background: white;
  padding: 20px;
  max-width: 300px;
  margin-inline: auto;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
  border-radius: 10px;
  font-size: 18px;
  line-height: 1.6;
}
</style>
</head>
<body>

<header>
  <h1>Multiplication Table Generator</h1>
</header>

<main>
  <p>Enter a number to see its multiplication table:</p>
  <input type="number" id="numberInput" placeholder="Enter a number" />
  <button onclick="generateTable()">Generate</button>

  <div class="output" id="tableOutput"></div>
</main>

<footer>
  <p>Made by Student | © 2025</p>
</footer>

<script>
function generateTable() {
  const number = document.getElementById('numberInput').value;
  const outputDiv = document.getElementById('tableOutput');
  outputDiv.innerHTML = "";
```

```

    if (!number) {
        outputDiv.innerHTML = 'Please enter a number!';
        return;
    }

    for (let i = 1; i <= 10; i++) {
        const result = `${number} × ${i} = ${number * i}`;
        const line = document.createElement('div');
        line.textContent = result;
        outputDiv.appendChild(line);
    }
}
</script>

</body>
</html>

```

18. Create a Java script code to generate a multiplication table asking the user to enter number of rows and columns. If user enters nothing or 0 then display multiplication table with 10 rows and 10 columns.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Dynamic Multiplication Table</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #eef2f7;
            margin: 0;
            padding: 0;
            text-align: center;
        }

        header, footer {
            background-color: #004080;
            color: white;
            padding: 15px;
        }
    </style>

```



```
main {
  padding: 30px;
}

input[type="number"] {
  padding: 10px;
  font-size: 16px;
  width: 100px;
  margin: 10px;
}

button {
  padding: 10px 20px;
  font-size: 16px;
  background-color: #004080;
  color: white;
  border: none;
  cursor: pointer;
  border-radius: 4px;
}

button:hover {
  background-color: #002d59;
}

table {
  margin: 20px auto;
  border-collapse: collapse;
}

td, th {
  border: 1px solid #333;
  padding: 8px 12px;
  background-color: white;
}

th {
  background-color: #cce0ff;
}
</style>
</head>
```

```
<body>

<header>
  <h1>Multiplication Table Generator</h1>
</header>

<main>
  <p>Enter number of rows and columns:</p>
  <input type="number" id="rows" placeholder="Rows">
  <input type="number" id="cols" placeholder="Columns">
  <button onclick="generateTable()">Generate Table</button>

  <div id="tableContainer"></div>
</main>
<footer>
  <p>Made by Student | © 2025</p>
</footer>

<script>
function generateTable() {
  let rows = parseInt(document.getElementById("rows").value);
  let cols = parseInt(document.getElementById("cols").value);

  if (!rows || rows <= 0) rows = 10;
  if (!cols || cols <= 0) cols = 10;

  let output = '<table><thead><tr><th>*</th>';
  for (let j = 1; j <= cols; j++) {
    output += `<th>${j}</th>`;
  }
  output += '</tr></thead><tbody>';

  for (let i = 1; i <= rows; i++) {
    output += `<tr><th>${i}</th>`;
    for (let j = 1; j <= cols; j++) {
      output += `<td>${i * j}</td>`;
    }
    output += '</tr>';
  }

  output += '</tbody></table>';
  document.getElementById("tableContainer").innerHTML = output;
}
```

```
}  
</script>
```

```
</body>  
</html>
```

19. Write a java script function which converts Fahrenheit to Celsius.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Fahrenheit to Celsius Converter</title>  
  <style>  
    body {  
      font-family: 'Segoe UI', sans-serif;  
      margin: 0;  
      padding: 0;  
      background: #f0f8ff;  
      display: flex;  
      flex-direction: column;  
      min-height: 100vh;  
    }  
  
    header, footer {  
      background-color: #0066cc;  
      color: white;  
      text-align: center;  
      padding: 20px;  
    }  
  
    main {  
      flex: 1;  
      padding: 40px 20px;  
      text-align: center;  
    }  
  
    h1 {  
      color: #003366;  
    }  
  
    input {  
      padding: 10px;  
      font-size: 16px;
```

```

width: 200px;
margin: 10px;
}

button {
padding: 10px 20px;
background-color: #0066cc;
color: white;
font-size: 16px;
border: none;
cursor: pointer;
border-radius: 5px;
}

button:hover {
background-color: #004c99;
}
#result {
margin-top: 20px;
font-size: 20px;
color: green;
font-weight: bold;
}
</style>
</head>
<body>

<header>
<h1>Temperature Converter</h1>
<p>Convert Fahrenheit to Celsius</p>
</header>

<main>
<h2>Enter temperature in Fahrenheit:</h2>
<input type="number" id="fahrenheit" placeholder="e.g. 98.6">
<br>
<button onclick="convert()">Convert</button>

<div id="result"></div>
</main>

<footer>

```

```
<p>Created by Student | © 2025</p>
</footer>
```

```
<script>
function fahrenheitToCelsius(f) {
    return ((f - 32) * 5 / 9).toFixed(2);
}

function convert() {
    const input = document.getElementById("fahrenheit").value;
    const resultDiv = document.getElementById("result");

    if (input === "" || isNaN(input)) {
        resultDiv.textContent = "Please enter a valid number!";
        resultDiv.style.color = "red";
        return;
    }
    const f = parseFloat(input);
    const c = fahrenheitToCelsius(f);
    resultDiv.textContent = `${f}°F = ${c}°C`;
    resultDiv.style.color = "green";
}
</script>
```

```
</body>
</html>
```

20. Write a JavaScript program which accept a number as input and insert dashes (-) between each two even numbers. For example if you accept 025468 the output should be 0-254-6-8.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Insert Dashes Between Even Numbers</title>
</head>
<body>

    <h2>Insert Dashes Between Even Numbers</h2>
    <input type="text" id="inputNumber" placeholder="Enter a number">
    <button onclick="insertDashes()">Submit</button>
```

```
<p>Output: <span id="output"></span></p>
```

```
<script>
function insertDashes() {
  let number = document.getElementById('inputNumber').value;
  let result = "";

  for (let i = 0; i < number.length; i++) {
    let currentDigit = number[i];
    let nextDigit = number[i + 1];
    result += currentDigit;
    if (isEven(currentDigit) && isEven(nextDigit)) {
      result += '-';
    }
  }

  document.getElementById('output').textContent = result;
}

function isEven(digit) {
  return digit % 2 === 0;
}
</script>
```

```
</body>
```

```
</html>
```

21. Write a JavaScript program to find the most frequent item of an array.

Sample array : `var arr1=[3, 'a', 'a', 'a', 2, 3, 'a', 3, 'a', 2, 4, 9, 3];`

Sample Output : a (5 times)

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Most Frequent Item Finder</title>
```

```
  <style>
```

```
    body {
```

```
      font-family: Arial, sans-serif;
```

```
      background-color: #f4f4f9;
```

```
      display: flex;
```

```
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
}
.container {
    background-color: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    text-align: center;
}
h1 {
    color: #333;
}
input[type="text"] {
    padding: 10px;
    width: 80%;
    margin: 10px 0;
    border: 1px solid #ccc;
    border-radius: 5px;
}
button {
    padding: 10px 20px;
    background-color: #4CAF50;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 16px;
}
button:hover {
    background-color: #45a049;
}
.result {
    margin-top: 20px;
    font-size: 18px;
    color: #333;
}
</style>
</head>
<body>
```

```

<div class="container">
  <h1>Find Most Frequent Item</h1>
  <p>Enter an array of numbers, separated by commas (e.g., 1, 2, 3, 3, 2):</p>
  <input type="text" id="arrayInput" placeholder="Enter array">
  <button onclick="findMostFrequentItem()">Find Most Frequent Item</button>
  <div class="result" id="result"></div>
</div>

<script>
function findMostFrequentItem() {
  let input = document.getElementById('arrayInput').value;
  let arr = input.split(',').map(item => item.trim());

  let frequencyMap = {};
  let maxFrequency = 0;
  let mostFrequentItem = null;

  for (let i = 0; i < arr.length; i++) {
    let item = arr[i];
    frequencyMap[item] = (frequencyMap[item] || 0) + 1;

    if (frequencyMap[item] > maxFrequency) {
      maxFrequency = frequencyMap[item];
      mostFrequentItem = item;
    }
  }

  document.getElementById('result').innerText = `Most Frequent Item:
${mostFrequentItem}`;
}
</script>

</body>
</html>

```

22. Write a Javascript code to display the following table to order Pizza on clicking confirm order following pop up box should get displayed

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

```



```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Pizza Order</title>
<style>
  table {
    width: 50%;
    border-collapse: collapse;
    margin: 20px 0;
  }
  th, td {
    border: 1px solid black;
    padding: 10px;
    text-align: center;
  }
  input {
    width: 50px;
    text-align: center;
  }
  button {
    margin: 10px;
    padding: 10px;
    cursor: pointer;
  }
</style>
</head>
<body>
```

```
<h2>Pizza Order Form</h2>
```

```
<table>
  <tr>
    <th>Item Name</th>
    <th>Price</th>
    <th>Quantity</th>
  </tr>
  <tr>
    <td>Chicken Pizza</td>
    <td>100</td>
    <td><input type="number" id="chicken" value="0"></td>
  </tr>
  <tr>
    <td>Paneer Pizza</td>
    <td>80</td>
```

```

        <td><input type="number" id="paneer" value="0"></td>
    </tr>
    <tr>
        <td>Veg Pizza</td>
        <td>70</td>
        <td><input type="number" id="veg" value="0"></td>
    </tr>
</table>

<label>Total Cost: </label>
<input type="text" id="totalCost" readonly><br>

<button onclick="calculateTotal()">Calculate Total</button>
<button onclick="confirmOrder()">Confirm Order</button>
<button onclick="cancelOrder()">Cancel Order</button>

<script>
    function calculateTotal() {
        let chickenQty = document.getElementById("chicken").value;
        let paneerQty = document.getElementById("paneer").value;
        let vegQty = document.getElementById("veg").value;

        let total = (chickenQty * 100) + (paneerQty * 80) + (vegQty * 70);
        document.getElementById("totalCost").value = total;
    }

    function confirmOrder() {
        let total = document.getElementById("totalCost").value;
        if (total > 0) {
            alert("Your order is confirmed! It will reach you in 10 minutes.");
        } else {
            alert("Please select at least one pizza to order.");
        }
    }

    function cancelOrder() {
        document.getElementById("chicken").value = 0;
        document.getElementById("paneer").value = 0;
        document.getElementById("veg").value = 0;
        document.getElementById("totalCost").value = "";
    }
</script>

```

```
</body>
</html>
```

23. Write a Javascript code to design a basic calculator given below.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Basic Calculator</title>
  <style>
    .calculator {
      width: 220px;
      margin: 50px auto;
      padding: 10px;
      border: 4px double black;
      background-color: #eee;
    }
    #display {
      width: 100%;
      height: 40px;
      font-size: 20px;
      margin-bottom: 10px;
      text-align: right;
      padding: 5px;
    }
    .buttons {
      display: grid;
      grid-template-columns: repeat(4, 1fr);
      gap: 5px;
    }
    .buttons button {
      padding: 15px;
      font-size: 18px;
      cursor: pointer;
    }
  </style>
</head>
<body>

  <div class="calculator">
    <input type="text" id="display" disabled>
```

```

<div class="buttons">
  <button onclick="appendValue('1')">1</button>
  <button onclick="appendValue('2')">2</button>
  <button onclick="appendValue('3')">3</button>
  <button onclick="appendValue('+')">+</button>

  <button onclick="appendValue('4')">4</button>
  <button onclick="appendValue('5')">5</button>
  <button onclick="appendValue('6')">6</button>
  <button onclick="appendValue('-')">-</button>

  <button onclick="appendValue('7')">7</button>
  <button onclick="appendValue('8')">8</button>
  <button onclick="appendValue('9')">9</button>
  <button onclick="appendValue('*')">*</button>

  <button onclick="clearDisplay()">C</button>
  <button onclick="appendValue('0')">0</button>
  <button onclick="calculate()">=</button>
  <button onclick="appendValue('/')">/</button>
</div>
</div>

<script>
function appendValue(val) {
  document.getElementById('display').value += val;
}

function clearDisplay() {
  document.getElementById('display').value = "";
}

function calculate() {
  try {
    let result = eval(document.getElementById('display').value);
    document.getElementById('display').value = result;
  } catch {
    document.getElementById('display').value = 'Error';
  }
}
</script>

```

```
</body>
</html>
```

24. IPv4 addresses are canonically represented in dot-decimal notation, which consists of four decimal numbers, each ranging from 0 to 255, separated by dots,e.g.,"172.16.254.1"

Frame a regular expression to check the given string is a IPv4 address or not.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>IPv4 Address Validator</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 40px;
    }
    input, button {
      padding: 8px;
      margin: 5px;
    }
    #result {
      font-weight: bold;
      margin-top: 10px;
    }
  </style>
</head>
<body>

  <h2>IPv4 Address Validator</h2>
  <label for="ip">Enter IP Address:</label>
  <input type="text" id="ip" placeholder="e.g., 192.168.0.1">
  <button onclick="validateIP()">Validate</button>
  <div id="result"></div>

  <script>
    function validateIP() {
      const ip = document.getElementById("ip").value;
      const ipv4Regex =
/^(25[0-5]|2[0-4]\d|1\d{2}|[1-9]?d)(\.(25[0-5]|2[0-4]\d|1\d{2}|[1-9]?d)){3}$/;
      const resultDiv = document.getElementById("result");
```

```

        if (ipv4Regex.test(ip)) {
            resultDiv.textContent = `"${ip}" is a valid IPv4 address.`;
            resultDiv.style.color = "green";
        } else {
            resultDiv.textContent = `"${ip}" is NOT a valid IPv4 address.`;
            resultDiv.style.color = "red";
        }
    }
}
</script>

</body>
</html>

```

25. Write a program to understand the difference between a real DOM and Virtual DOM.

Real DOM (Vanilla JS):

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Real DOM Example</title>
</head>
<body>

  <h2>Real DOM Counter</h2>
  <p id="realCounter">0</p>
  <button onclick="incrementReal()">Increment</button>

  <script>
    let realCount = 0;
    function incrementReal() {
      realCount++;
      document.getElementById("realCounter").innerText = realCount;
    }
  </script>

</body>
</html>

```

Virtual DOM (React JS):

```

import React, { useState } from 'react';
import ReactDOM from 'react-dom';

```

```

function VirtualDOMCounter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Virtual DOM Counter</h2>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

ReactDOM.render(<VirtualDOMCounter />, document.getElementById('root'));

```

26. Write a program to demonstrate the React functional Component.

```

// App.js
import React from 'react';

// ✅ Functional Component
function Welcome(props) {
  return <h2>Hello, {props.name}!</h2>;
}

// ✅ Another Functional Component using state
function Counter() {
  const [count, setCount] = React.useState(0);

  return (
    <div>
      <h3>Counter: {count}</h3>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

// ✅ Main App Component
function App() {
  return (
    <div style={{ textAlign: 'center', marginTop: '40px' }}>
      <Welcome name="Greeshma" />
    </div>
  );
}

```

```

        <Counter />
    </div>
  );
}

```

```
export default App;
```

27. Write a program to demonstrate calling a component from another component.

```
// App.js
```

```
import React from 'react';
```

```
//  Child Component
```

```
function ChildComponent() {
  return <p>I am the Child Component!</p>;
}
```

```
//  Parent Component
```

```
function ParentComponent() {
  return (
    <div style={{ textAlign: 'center', marginTop: '40px' }}>
      <h2>I am the Parent Component</h2>
      { /* Calling Child Component */ }
      <ChildComponent />
    </div>
  );
}
```

```
//  Main App Component
```

```
function App() {
  return (
    <div>
      <ParentComponent />
    </div>
  );
}
```

```
export default App;
```

28. Write a program to demonstrate React Class Components.

```
// App.js
```

```
import React, { Component } from 'react';
```


//  Class Component

```
class Welcome extends Component {  
  render() {  
    return <h2>Hello, {this.props.name}!</h2>;  
  }  
}
```

//  Another Class Component with State

```
class Counter extends Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      count: 0  
    };  
  }  
  
  increment = () => {  
    this.setState({ count: this.state.count + 1 });  
  }  
  
  render() {  
    return (  
      <div>  
        <h3>Counter: {this.state.count}</h3>  
        <button onClick={this.increment}>Increment</button>  
      </div>  
    );  
  }  
}
```

//  Main App Component

```
class App extends Component {  
  render() {  
    return (  
      <div style={{ textAlign: 'center', marginTop: '40px' }}>  
        <Welcome name="Saneela" />  
        <Counter />  
      </div>  
    );  
  }  
}
```

```
export default App;
```

29. Write a program to demonstrate React props.

```
// App.js
```

```
import React from 'react';
```

```
//  Child component using props
```

```
function Message(props) {
```

```
  return (
```

```
    <div>
```

```
      <h2>Hello, {props.name}!</h2>
```

```
      <p>Your role is: {props.role}</p>
```

```
    </div>
```

```
  );
```

```
}
```

```
//  Parent component
```

```
function App() {
```

```
  return (
```

```
    <div style={{ textAlign: 'center', marginTop: '50px' }}>
```

```
      { /* Passing props to the child */ }
```

```
      <Message name="Greeshma" role="Frontend Developer Intern" />
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

30) Write code to illustrate the lifecycle of React JS.

Code:

```
import React from "react";
class LifecycleDemo extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0,
    };
    console.log("Constructor: Component is initialized.");
  }

  componentDidMount() {
    console.log("componentDidMount: Component is mounted to the DOM.");
  }

  componentDidUpdate(prevProps, prevState) {
    if (prevState.count !== this.state.count) {
      console.log("componentDidUpdate: Component updated.");
    }
  }

  componentWillUnmount() {
    console.log("componentWillUnmount: Component is about to be removed.");
  }

  incrementCount = () => {
    this.setState((prevState) => ({ count: prevState.count + 1 }));
  };

  render() {
    console.log("Render: Component is rendering.");
    return (
      <div>
        <h1>React Lifecycle Demo</h1>
        <p>Count: {this.state.count}</p>
        <button onClick={this.incrementCount}>Increment</button>
      </div>
    );
  }
}

export default LifecycleDemo;
```

Output:

React Lifecycle Demo

Count: 1

Increment

Constructor: Component is initialized.	LifecycleDemo.jsx:9
Constructor: Component is initialized.	hook.js:377
Render: Component is rendering.	LifecycleDemo.jsx:31
Render: Component is rendering.	hook.js:377
componentDidMount: Component is mounted to the DOM.	LifecycleDemo.jsx:13
componentWillUnmount: Component is about to be removed.	hook.js:377
componentDidMount: Component is mounted to the DOM.	hook.js:377
Render: Component is rendering.	LifecycleDemo.jsx:31
Render: Component is rendering.	LifecycleDemo.jsx:31
componentDidUpdate: Component updated.	LifecycleDemo.jsx:18

31. Write a program to demonstrate useState Hook.

Code:

```
import React, { useState } from "react";
function Counter() {
  // Declare a state variable "count" and a function "setCount" to update
  const [count, setCount] = useState(0);

  return (
    <div style={{ textAlign: "center", marginTop: "2rem" }}>
      <h1>useState Hook Example</h1>
      <p>Current Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>{" "}
      <button onClick={() => setCount(count - 1)}>Decrement</button>{" "}
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
export default Counter;
```

Output:

useState Hook Example

Current Count: 3

Increment

Decrement

Reset

useState Hook Example

Current Count: 2

Increment

Decrement

Reset

useState Hook Example

Current Count: 0

Increment

Decrement

Reset

32 Write a program to demonstrate useEffect Hook.

Code:

```
import React, { useState, useEffect } from "react";
function UseEffectDemo() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    console.log("Component mounted or count updated:", count);
    return () => {
      console.log("Cleanup before the next effect or unmount.");
    };
  }, [count]);
  return (
    <div style={{ textAlign: "center", marginTop: "2rem" }}>
      <h1>useEffect Hook Example</h1>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
export default UseEffectDemo;
```

Output:**useEffect Hook Example**

Count: 1

Increment

Component mounted or count updated: 0

Cleanup before the next effect or unmount.

Component mounted or count updated: 0

Cleanup before the next effect or unmount.

Component mounted or count updated: 1

33. Write a program to demonstrate useContext and useRef Hook.**Code:****Create a context (UserContext.jsx)**

```
import React, { createContext, useState } from "react";
export const UserContext = createContext();
export function UserProvider({ children }) {
  const [userName, setUserName] = useState("");
  return (
    <UserContext.Provider value={{ userName, setUserName }}>
      {children}
    </UserContext.Provider>
  );
}
```

స్వయం తేజస్విన్ భవ

1979

Create a component that uses useContext and useRef (UserForm.jsx)

```
import React, { useContext, useRef } from "react";
import { UserContext } from "../UserContext";
function UserForm() {
  const { userName, setUserName } = useContext(UserContext);
  const inputRef = useRef(null);
  const handleSubmit = (e) => {
    e.preventDefault();
    setUserName(inputRef.current.value);
    inputRef.current.value = "";
    inputRef.current.focus();
  };
  return (
    <div style={{ textAlign: "center", marginTop: "2rem" }}>
      <h2>Hello, {userName || "Guest"}!</h2>
      <form onSubmit={handleSubmit}>
        <input
          ref={inputRef}
          type="text"
          placeholder="Enter your name"
          style={{ padding: "8px", fontSize: "16px" }}
        />
        <button type="submit" style={{ marginLeft: "10px", padding: "8px" }}>
          Submit
        </button>
      </form>
    </div>
  );
}
export default UserForm;
```

Output:

Before Entering name:

Hello, Guest!

After Entering Name:

Hello, Moukthika!

స్వయం తేజస్విన్ భవ

1979

34. Write a program to demonstrate custom Hook.**Code:**

Create the custom hook (useWindowWidth.js)

```
import { useState, useEffect } from "react";
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener("resize", handleResize);
    return () => {
      window.removeEventListener("resize", handleResize);
    };
  }, []);
  return width;
}
export default useWindowWidth;
```

Use the custom hook in a component (WindowWidthDisplay.jsx)

```
import React from "react";
import useWindowWidth from "../useWindowWidth";
function WindowWidthDisplay() {
  const width = useWindowWidth();
  return (
    <div style={{ textAlign: "center", marginTop: "2rem" }}>
      <h1>Custom Hook Demo: useWindowWidth</h1>
      <p>Current window width: {width}px</p>
    </div>
  );
}
export default WindowWidthDisplay;
```


Output:**Full Screen Width:**

Custom Hook Demo: useWindowWidth

Current window width: 1396px

After Changing Width:

Custom Hook Demo: useWindowWidth

Current window width: 722px

35. Implement Routing in React JS.**Code:****Home.jsx**

```
import React from "react";  
function Home() {  
  return <h1>🏠 Home Page</h1>;  
}  
export default Home;
```

About.jsx

```
import React from "react";  
function About() {  
  return <h1>📄 About Page</h1>;  
}  
export default About;
```

Contact.jsx

```
import React from "react";
function Contact() {
  return <h1>📞 Contact Page</h1>;
}
export default Contact;
```

App.jsx

```
import Contact from "../pages/contact";
function App (){
  return (
    <div>
      <BrowserRouter>
        <nav style={{ padding: "1rem", borderBottom: "1px solid #ccc" }}>
          <Link to="/" style={{ margin: "0 10px" }}>Home</Link>
          <Link to="/about" style={{ margin: "0 10px" }}>About</Link>
          <Link to="/contact" style={{ margin: "0 10px" }}>Contact</Link>
        </nav>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/contact" element={<Contact />} />
        </Routes>
      </BrowserRouter>
    </div>
  );
}
```

Output:

[Home](#) [About](#) [Contact](#)



Home Page

[Home](#) [About](#) [Contact](#)



About Page

[Home](#) [About](#) [Contact](#)



Contact Page

36 Design a college admission enquiry form and store details in mongoDB using states and events as a React Functional Component.**Code:****Frontend – React Form:**

```
import React, { useState } from "react";
function AdmissionForm() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    phone: "",
    course: "",
    message: "",
  });
  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };
  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const response = await fetch("http://localhost:5000/api/enquiry", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(formData),
      });
      const data = await response.json();
      alert(data.message);
    } catch (error) {
      alert("Submission failed!");
    }
  };
  return (
    <form onSubmit={handleSubmit} style={{ padding: "2rem", maxWidth: "500px", margin: "auto" }}>
      <h2>College Admission Enquiry</h2>
      <input type="text" name="name" placeholder="Full Name" required onChange={handleChange} /><br /><br />
      <input type="email" name="email" placeholder="Email" required onChange={handleChange} /><br /><br />
      <input type="tel" name="phone" placeholder="Phone Number" required onChange={handleChange} /><br /><br />
      <input type="text" name="course" placeholder="Course Interested In" required onChange={handleChange} /><br /><br />
      <textarea name="message" placeholder="Your Message" rows="4" onChange={handleChange}></textarea><br /><br />
      <button type="submit">Submit</button>
    </form>
  );
}
export default AdmissionForm;
```

Server.js

```
import express, { json } from "express";
import { connect, Schema, model } from "mongoose";
import cors from "cors";
const app = express();
app.use(cors());
app.use(json());
connect("mongodb://127.0.0.1:27017/collegeEnquiry", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
const enquirySchema = new Schema({
  name: String,
  email: String,
  phone: String,
  course: String,
  message: String,
});
const Enquiry = model("Enquiry", enquirySchema);
app.post("/api/enquiry", async (req, res) => {
  try {
    const newEnquiry = new Enquiry(req.body);
    await newEnquiry.save();
    res.json({ message: "Enquiry submitted successfully!" });
  } catch (error) {
    res.status(500).json({ message: "Error submitting enquiry" });
  }
});
app.listen(5000, () => {
  console.log("Server is running on http://localhost:5000");
});
```

Output:

37 Develop a CRUD Application using MERN.

Code:

Frontend – React Form:

```
import React, { useState } from "react";
function AdmissionForm() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    phone: "",
    course: "",
    message: "",
  });
  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };
  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const response = await fetch("http://localhost:5000/api/enquiry", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(formData),
      });
      const data = await response.json();
      alert(data.message);
    } catch (error) {
      alert("Submission failed!");
    }
  };
  return (
    <form onSubmit={handleSubmit} style={{ padding: "2rem", maxWidth: "500px", margin: "auto" }}>
      <h2>College Admission Enquiry</h2>
      <input type="text" name="name" placeholder="Full Name" required onChange={handleChange} /><br /><br />
      <input type="email" name="email" placeholder="Email" required onChange={handleChange} /><br /><br />
      <input type="tel" name="phone" placeholder="Phone Number" required onChange={handleChange} /><br /><br />
      <input type="text" name="course" placeholder="Course Interested In" required onChange={handleChange} /><br /><br />
      <textarea name="message" placeholder="Your Message" rows="4" onChange={handleChange}></textarea><br /><br />
      <button type="submit">Submit</button>
    </form>
  );
}
export default AdmissionForm;
```

Server.js

```
import express, { json } from "express";
import { connect, Schema, model } from "mongoose";
import cors from "cors";
const app = express();
app.use(cors());
app.use(json());
connect("mongodb://127.0.0.1:27017/collegeEnquiry", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
const enquirySchema = new Schema({
  name: String,
  email: String,
  phone: String,
  course: String,
  message: String,
});
const Enquiry = model("Enquiry", enquirySchema);
app.post("/api/enquiry", async (req, res) => {
  try {
    const newEnquiry = new Enquiry(req.body);
    await newEnquiry.save();
    res.json({ message: "Enquiry submitted successfully!" });
  } catch (error) {
    res.status(500).json({ message: "Error submitting enquiry" });
  }
});
app.listen(5000, () => {
  console.log("Server is running on http://localhost:5000");
});
```

Output:

localhost:5173 says
Enquiry submitted successfully!

OK

College Admission Enquiry

TULASI SREE MOUKTHIK

tulasisreemoukthika2005@i

9346091348

WT

Hi

Submit

- CBIT
- admin
- collegeEnquiry
 - enquiries
- config
- local

```
_id: ObjectId('680a434a8fbc62af602ceff9')
name: "TULASI SREE MOUKTHIKA "
email: "tulasisreemoukthika2005@gmail.com"
phone: "9346091348"
course: "WT"
message: "Hi"
__v: 0
```

38 Develop an Attendance Management Module for student attendance entry and Verifying attendance by students using MongoDB, Express JS, React JS and Node JS (MERN).

Code:

Backend:Attendance.js

```
const mongoose = require("mongoose");

const attendanceSchema = new mongoose.Schema({
  studentId: String,
  name: String,
  course: String,
  date: String,
  status: String // Present / Absent
});

module.exports = mongoose.model("Attendance", attendanceSchema);
```

attendanceRoutes.js

```
const express = require("express");
const router = express.Router();
const Attendance = require("../models/Attendance");
router.post("/", async (req, res) => {
  try {
    const record = new Attendance(req.body);
    await record.save();
    res.json({ message: "Attendance recorded!" });
  } catch (err) {
    res.status(500).json({ message: "Error saving attendance" });
  }
});
router.get("/:studentId", async (req, res) => {
  try {
    const records = await Attendance.find({ studentId: req.params.studentId });
    res.json(records);
  } catch (err) {
    res.status(500).json({ message: "Error fetching records" });
  }
});
module.exports = router;
```

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const attendanceRoutes = require("./routes/attendanceRoutes");
const app = express();
app.use(cors());
app.use(express.json());
mongoose.connect("mongodb://127.0.0.1:27017/attendanceDB", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
}).then(() => console.log("MongoDB connected"));
app.use("/api/attendance", attendanceRoutes);
app.listen(5000, () => console.log("Server started on http://localhost:5000));
```


Frontend:App.jsx

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [form, setForm] = useState({ studentId: "", name: "", course: "", date: "", status: "Present" });
  const [searchId, setSearchId] = useState("");
  const [records, setRecords] = useState([]);
  const handleAdd = async (e) => {
    e.preventDefault();
    await axios.post("http://localhost:5000/api/attendance", form);
    alert("Attendance added!");
    setForm({ studentId: "", name: "", course: "", date: "", status: "Present" });
  };
  const handleSearch = async () => {
    const res = await axios.get(`http://localhost:5000/api/attendance/${searchId}`);
    setRecords(res.data);
  };
  return (
    <div style={{ padding: "2rem" }}>
      <h2>Attendance Management</h2>
      <h3>Add Attendance</h3>
      <form onSubmit={handleAdd}>
        <input placeholder="Student ID" value={form.studentId} onChange={e => setForm({ ...form, studentId: e.target.value })} required />
        <input placeholder="Name" value={form.name} onChange={e => setForm({ ...form, name: e.target.value })} required />
        <input placeholder="Course" value={form.course} onChange={e => setForm({ ...form, course: e.target.value })} required />
        <input type="date" value={form.date} onChange={e => setForm({ ...form, date: e.target.value })} required />
        <select value={form.status} onChange={e => setForm({ ...form, status: e.target.value })}>
          <option value="Present">Present</option>
          <option value="Absent">Absent</option>
        </select>
        <button type="submit">Submit</button>
      </form>
      <h3>Verify Attendance</h3>
      <input placeholder="Enter Student ID" value={searchId} onChange={e => setSearchId(e.target.value)} />
      <button onClick={handleSearch}>Check</button>
      <ul>
        {records.map((rec, idx) => (
          <li key={idx}>
            {rec.date} | {rec.course} | {rec.status}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

Output:

INSTITUTE OF TECHNOLOGY

Attendance Management**Add Attendance**

Student ID	Name	Course	dd-mm-yyyy	Present	Submit
---	-----------------------------------	-------------------------------------	---	--------------------------------------	---------------------------------------

Verify Attendance

Enter Student ID	Check
---	--------------------------------------

localhost:3173 says
Attendance added!
OK

Attendance Management

Add Attendance

160123733176	Moukthika	WT	23 - 04 - 2025	Present ▾	Submit
--------------	-----------	----	----------------	-----------	--------

Verify Attendance

- 2025-04-23 | WT | Present

MongoDB:

```
_id: ObjectId('680a4f21a8bcc6f4e3d497a6')
studentId : "160123733176"
name : "Moukthika"
course : "WT"
date : "2025-04-23"
status : "Present"
__v : 0
```